

ROS2

DAY 4 – Manual 2

ROS2 Services – Server & Client Implementation

Objective

By the end of this manual, you will be able to:

1. Write a ROS2 Service Server node (srv_server.py) in my_py_pkg.
 2. Write a ROS2 Service Client node (srv_client.py) in my_py_pkg.
 3. Understand each line of the Python code.
 4. Run and test a ROS2 service using TestService.srv.
-

Step 0 – Pre-Requisites

- Manual 1 completed (workspace built, .srv ready).
- ROS2 workspace created, my_py_pkg (package) created and sourced:

```
source ~/ros2_ws/install/setup.bash
```

Step 1 – Create Service Server Node

1. Navigate to the Python package folder:

```
cd ~/ros2_ws/src/my_py_pkg/my_py_pkg
```

2. Create the server node file:

```
nano srv_server.py
```

3. Paste the code:

```
import rclpy
from rclpy.node import Node
from service_pkg.srv import TestService
```

```
class ServiceServer(Node):
    def __init__(self):
```

```

super().__init__('ServiceServerNode')
self.srv = self.create_service(TestService, 'test_service', self.callback)
self.get_logger().info("Service Server Ready: test_service")

def callback(self, request, response):
    self.get_logger().info(f"Incoming request: {request.input}")
    response.output = "hello client... Got your req"
    self.get_logger().info(f"Sending response: {response.output}")
    return response

def main(args=None):
    rclpy.init(args=args)
    node = ServiceServer()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Explanation – Server Node

Line	Explanation
import rclpy	ROS2 Python client library.
from rclpy.node import Node	Base class for creating ROS2 nodes.
from service_pkg.srv import TestService	Imports the generated Python service from .srv file.
class ServiceServer(Node)	Creates a ROS2 node called ServiceServerNode.
self.create_service(TestService, 'test_service', self.callback)	Registers the service with ROS2; 'test_service' is the service name; callback handles requests.
self.get_logger().info()	Prints info messages in the terminal.
callback()	Function called when client sends a request; reads request, sets response.
rclpy.spin(node)	Keeps server running.
rclpy.shutdown()	Gracefully shuts down ROS2.

Step 2 – Create Service Client Node

1. In the same folder:

nano srv_client.py

2. Paste the code:

```
import rclpy
from rclpy.node import Node
from service_pkg.srv import TestService

class ServiceClient(Node):
    def __init__(self):
        super().__init__('ServiceClientNode')
        self.client = self.create_client(TestService, 'test_service')
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().warn("Service not available... waiting")

    def send_request(self, req):
        request = TestService.Request()
        request.input = req
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

def main(args=None):
    rclpy.init(args=args)
    client = ServiceClient()
    response = client.send_request("Hello server")
    print(f"Result from server: {response.output}")
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Explanation – Client Node

Line	Explanation
self.create_client(TestService, 'test_service')	Connects to the server service.
while not self.client.wait_for_service(timeout_sec=1.0):	Waits until server is active; prints warning if not ready.
request = TestService.Request()	Prepares request object.
request.input = req	Sets the string to send.
future = self.client.call_async(request)	Sends request to server asynchronously.

<code>rclpy.spin_until_future_complete(self, future)</code>	Waits for server to respond.
<code>future.result()</code>	Gets the response object.

Step 3 – Update setup.py for Nodes

In `my_py_pkg/setup.py`:

```
entry_points={
    'console_scripts': [
        'srv_server = my_py_pkg.srv_server:main',
        'srv_client = my_py_pkg.srv_client:main',
    ],
}
```

- This allows:

```
ros2 run my_py_pkg srv_server
ros2 run my_py_pkg srv_client
```

Step 4 – Build Workspace

```
cd ~/ros2_ws
colcon build --packages-select my_py_pkg
source install/setup.bash
```

Step 5 – Run the Service

5.1 Start Server

```
ros2 run my_py_pkg srv_server
```

Expected output:

```
[INFO] [ServiceServerNode]: Service Server Ready: test_service
```

5.2 Start Client

Open a new terminal:

```
ros2 run my_py_pkg srv_client
```

Expected output:

Result from server: hello client... Got your req

Server terminal will show:

[INFO] [ServiceServerNode]: Incoming request: Hello server

[INFO] [ServiceServerNode]: Sending response: hello client... Got your req

5.3 Test Multiple Requests

Modify the client code to send other strings:

```
response = client.send_request("Reset Motors")
```

Server will respond accordingly:

Server terminal: Incoming request: Reset Motors

Client output: Result from server: hello client... Got your req

Step 6 – Concepts Recap

- Services = synchronous request/response
 - Server node waits for requests
 - Client node sends requests and waits
 - .srv file defines the request and response structure
 - Python nodes are registered in setup.py for ros2 run
 - Learners can now build, run, and test ROS2 services
-

✓ Manual 2 Complete!

Your my_py_pkg is ready with working server and client nodes, fully aligned with TestService.srv.
